

Laboratory 4 Report

ANDREA BOTTICELLA*, s347291, Politecnico di Torino, Italy

ELIA INNOCENTI*, s345388, Politecnico di Torino, Italy

SIMONE ROMANO*, s344024, Politecnico di Torino, Italy

This laboratory studies malicious SSH sessions as sequences of Bash entities and their associated MITRE ATT&CK tactics. We first characterize the labelled dataset and examine how tactic assignment depends on command context. We then compare a general-purpose BERT tokenizer with the code-oriented UniXcoder tokenizer, quantify their behavior on shell commands and complete sessions, and apply word truncation to control pathological token growth. Subsequent stages fine-tune and compare transformer models for tactic prediction and use the best configuration to analyse unlabelled honeypot sessions.

1 INTRODUCTION

SSH honeypots record commands, flags, parameters, and separators entered by remote users. In this laboratory, each space-separated Bash entity is treated as an SSH word and is associated with a MITRE tactic describing the adversary's intent. The objective is to build a Named Entity Recognition pipeline that can learn these contextual labels from a small annotated dataset and later support the analysis of unlabelled sessions.

The workflow comprises four stages: **dataset characterization**, **tokenization analysis**, **transformer fine-tuning**, and **inference with tactic fingerprints**. This report follows the same order. All payloads are treated strictly as text: long encoded strings are truncated when required and are never decoded or executed.

2 TASK 1: DATASET CHARACTERIZATION

The labelled data comprise **251 training sessions** and **108 test sessions**. Each session is represented by a raw command string and a list of MITRE tactics aligned with its space-separated Bash words.

2.1 Label Distribution

Distinct tactics and imbalance. The dataset contains seven tactics: Discovery, Execution, Persistence, Impact, Defense Evasion, Not Malicious Yet, and Other. Figure 1 reports their word-level frequencies in both partitions. The two distributions have a similar shape: Discovery dominates with **52.37%** of training words and **53.36%** of test words, followed by Execution (**28.23%/25.30%**) and Persistence (**9.87%/11.02%**). Every remaining class accounts for less than 4% of either split.

This imbalance is important for the modelling stage: accuracy alone will be insufficient, because a classifier can perform well on the dominant tactics while neglecting minority behaviors such as Impact or Other. Macro-averaged metrics and per-class F1 scores are therefore necessary.

2.2 Contextual Meaning of echo

Observed labels. In the training set, echo occurs with six different tactics: Persistence (104 occurrences), Execution (39), Discovery (31), Not Malicious Yet (8), Impact (6), and Other (4). A command name is therefore not sufficient to determine its tactic.

Persistence example. In the sequence `echo root:<password> | chpasswd | bash`, the emitted string is consumed by `chpasswd` to modify credentials. The command contributes to maintaining access and is consequently labelled Persistence.

Execution example. In `echo <encoded payload> | base64 -decode | bash`, the output is decoded and passed directly to a shell. Here, echo participates in launching a payload and is labelled Execution. These examples show that tactic assignment depends on neighboring words and downstream use, motivating a contextual sequence model rather than a command-to-label lookup table.

2.3 Session Lengths

Length variability. Training sessions range from 2 to 224 Bash words, with mean **45.70** and median **17**. Test sessions have the same range but are generally longer, with mean **57.38** and median **23.5**. The large gap between means and medians indicates a right-skewed distribution: many sessions are short, while a small number of long traces extend the tail.

Figure 2 visualizes the empirical cumulative distribution over the labelled sessions. The long tail is relevant because the number of Bash words does not directly equal the number of model tokens: unusual shell strings can be fragmented into many subwords and exceed a transformer's context limit even when the word count appears manageable.

*The authors collaborated closely in developing this project.

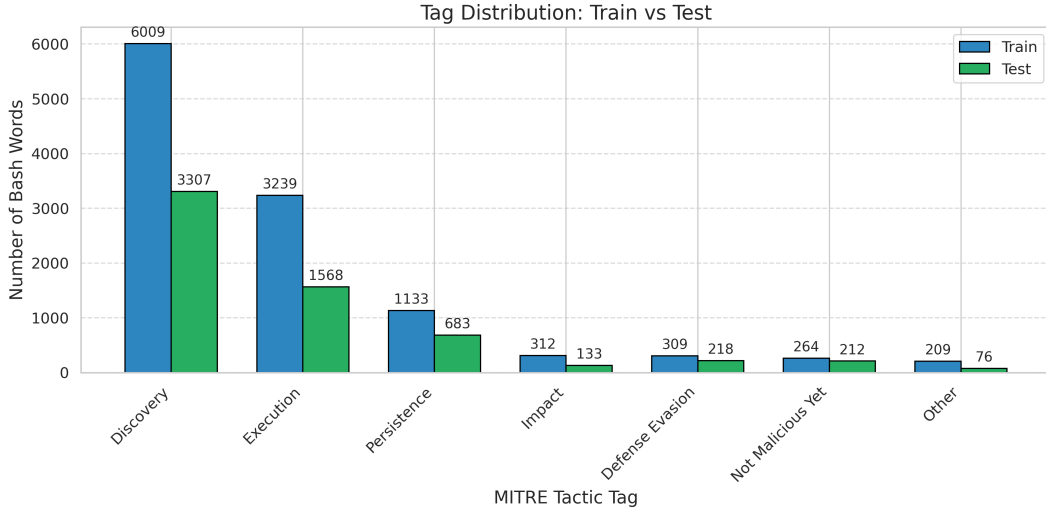


Fig. 1. Distribution of MITRE tactic labels over Bash words in the training and test partitions.

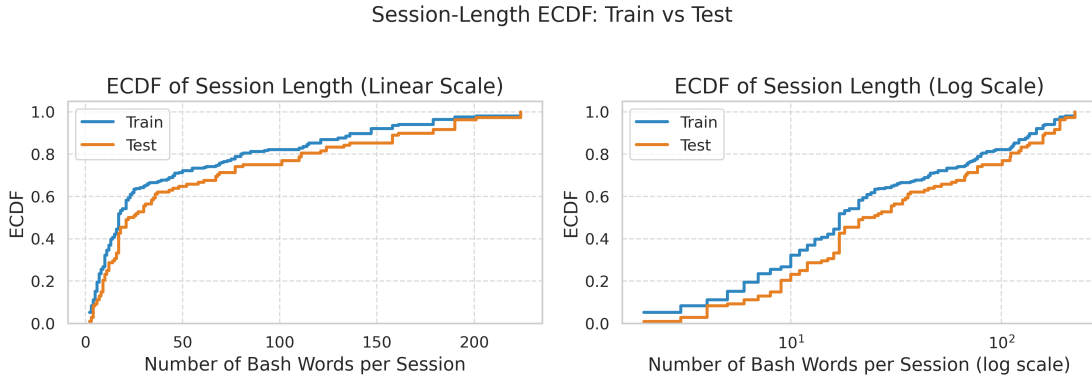


Fig. 2. ECDF of the number of Bash words per labelled SSH session in the training and test partitions, shown on linear and logarithmic scales.

3 TASK 2: TOKENIZATION

We compare **BERT-base uncased**, trained primarily on natural-language text, and **UniXcoder-base**, whose pre-training also includes source code. Both tokenizers are evaluated with a maximum sequence length of 512.

3.1 Tokenization of SSH Commands

Command-level comparison. Table 1 shows the subword decomposition of the nine required commands. Prefix markers used by the tokenizers are omitted from the table for readability. BERT uses ## for continuation pieces, whereas UniXcoder uses a spacing marker on word-initial tokens.

BERT produces 16 tokens for the nine words, a token-to-word ratio of **1.78**; UniXcoder produces 14, yielding the better ratio of **1.56**. Common words such as `cat`, `echo`, and `shell` are retained by both vocabularies because they are sufficiently frequent in their pre-training corpora. UniXcoder additionally preserves `grep`, consistent with its exposure to code and command-line text.

3.2 Full-Corpus Tokenization and Context Limit

Token counts before preprocessing. Table 2 summarizes tokenization of all 251 training sessions. Although UniXcoder has a lower median, its mean and maximum are much larger because rare shell strings are decomposed rather than collapsed into unknown tokens. Across the corpus, BERT emits **303 [UNK] tokens**, whereas UniXcoder emits none.

Consequently, **24 BERT sessions (9.56%)** and **29 UniXcoder sessions (11.55%)** exceed 512 tokens. Figure 3 shows the corresponding ECDFs and the context threshold. Most sessions remain below the limit, but UniXcoder has a visibly long tail.

Longest session. Both maxima occur on the same **134-word** training session. Two words exceed 30 characters; the longest is a **46,860-character encoded payload**, which is kept strictly as text. BERT replaces portions of the rare sequence with [UNK] and produces 1,889 tokens, whereas UniXcoder decomposes it into 28,920 tokens. The code-oriented tokenizer is therefore compact on ordinary commands but much more verbose on pathological strings.

Table 1. Subword tokenization of the selected SSH commands. Table 2. Training-corpus tokenization before word truncation.

Command	BERT	UniXcoder
cat	cat	cat
shell	shell	shell
echo	echo	echo
top	top	top
chpasswd	ch, pass, wd	ch, passwd
crontab	cr, ont, ab	c, ront, ab
wget	w, get	w, get
busybox	busy, box	busy, box
grep	gr, ep	grep

Metric	BERT	UniXcoder
Mean tokens/session	178.59	409.32
Median tokens/session	71	62
Maximum tokens/session	1,889	28,920
Corpus [UNK] count	303	0
Sessions above 512	24	29

Tokenization Distributions and Truncation Analysis

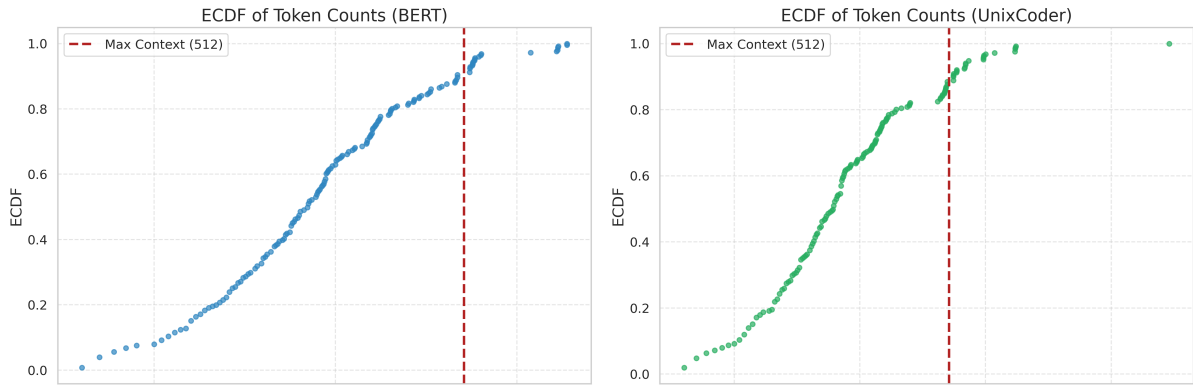


Fig. 3. ECDF of token counts before preprocessing. The dashed line marks the 512-token context limit.

Words vs Tokens (After Word Truncation)

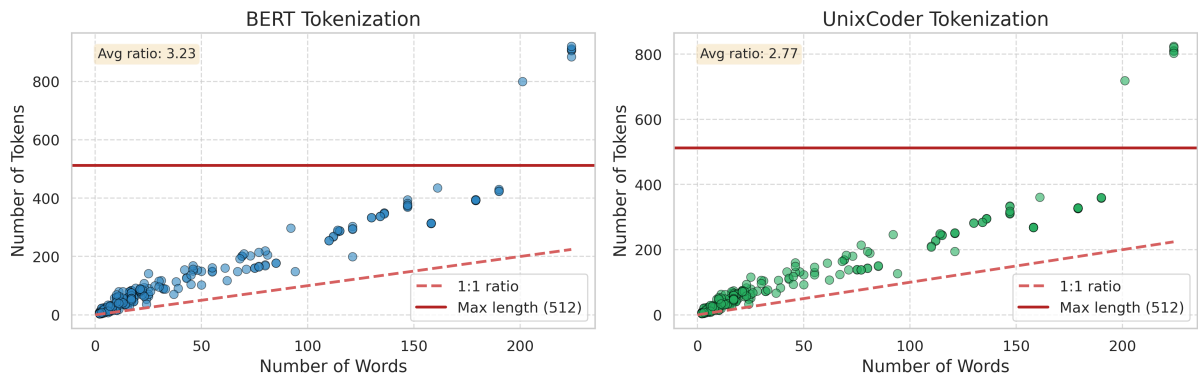


Fig. 4. Number of Bash words versus generated tokens after truncating individual words to 30 characters.

3.3 Truncating Long Bash Words

Preprocessing effect. Following the prescribed preprocessing, every Bash word longer than 30 characters is shortened before tokenization. This reduces the average sequence length to **128.38 tokens for BERT** and **110.52 for UniXcoder**; the maxima fall to 920 and 824, respectively. Only **six sessions per tokenizer** remain above the 512-token limit.

Figure 4 relates Bash-word counts to token counts after preprocessing. Using the notebook’s per-session definition $\frac{1}{N} \sum_i \frac{\text{tokens}_i}{\text{words}_i}$, the average token-to-word ratio is **3.23 for BERT** and **2.77 for UniXcoder**. Thus, once abnormally long words are controlled, UniXcoder provides the more compact representation of typical SSH syntax.

4 TASK 3: MODEL TRAINING

The NER task is formulated as word-level MITRE tactic prediction after subword tokenization. Labels are aligned with the first token of each Bash word, while continuation pieces are ignored by the loss and metrics. The original training split is further divided into **202 training sessions** and **49 validation sessions** using multilabel stratification at session level, so that rare tactics are represented more reliably during model selection. The held-out test set contains **108 sessions**.

4.1 Validation Strategy and Fine-Tuning Setup

For the full fine-tuning experiments, model selection is based on validation macro F1 together with validation loss. Macro F1 is the main classification criterion because the labelled dataset is imbalanced, while validation loss is used to detect unstable convergence or late overfitting. All final numbers in the tables are computed on the held-out test set only after selecting the configuration from the validation histories.

Pre-trained BERT. The first model is bert-base-uncased fine-tuned end-to-end for token classification. Despite the small labelled dataset, the model reaches **84.76% token accuracy**, **61.97% macro F1**, and **79.97% average session fidelity** on the test set (Table 3). This is a useful result, but the gap between accuracy and macro F1 shows that the model is mostly strong on frequent classes and less reliable on minority tactics.

Figure 5 shows the validation behaviour for the BERT learning-rate search. The smallest rate is stable but slow, while larger rates improve faster and then become more irregular. The selected checkpoint is therefore not simply the last epoch, but the epoch that gives the best validation compromise before the model starts fitting the small training split too closely.

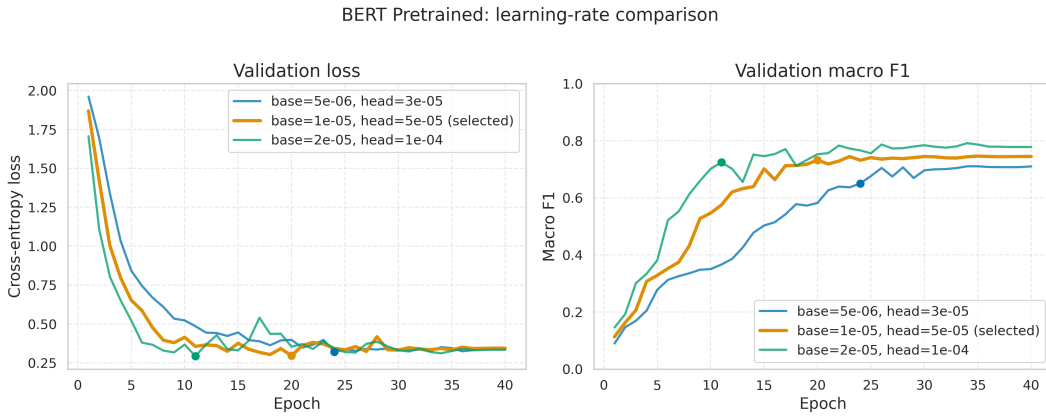


Fig. 5. Validation loss and macro F1 curves for pre-trained BERT across learning rates.

Naked BERT baseline. To isolate the contribution of pre-training, the same BERT architecture is trained from random initialization. Its macro F1 drops to **44.17%**, confirming that the task is not solved equally well by any large architecture when only 251 labelled sessions are available. Pre-training therefore provides a strong prior, even though it was learned mostly on natural language rather than shell logs.

UnixCoder. The code-oriented microsoft/unixcoder-base model is the best full fine-tuned model. It reaches **87.10% accuracy**, **73.50% macro F1**, and **86.29% fidelity**, improving over pre-trained BERT by **11.53 macro-F1 points**. This supports the hypothesis that a model pre-trained on code-like data is better suited to SSH sessions, which contain commands, paths, flags, separators, and encoded strings rather than natural prose.

The UnixCoder validation curves in Figure 6 show a clearer separation between useful and less useful learning rates. The selected configuration reaches the best validation macro F1 without requiring the very late epochs, which is important because the training set has only a few hundred sessions. This behaviour supports using UnixCoder as the backbone for the subsequent frozen-layer experiments and for inference.

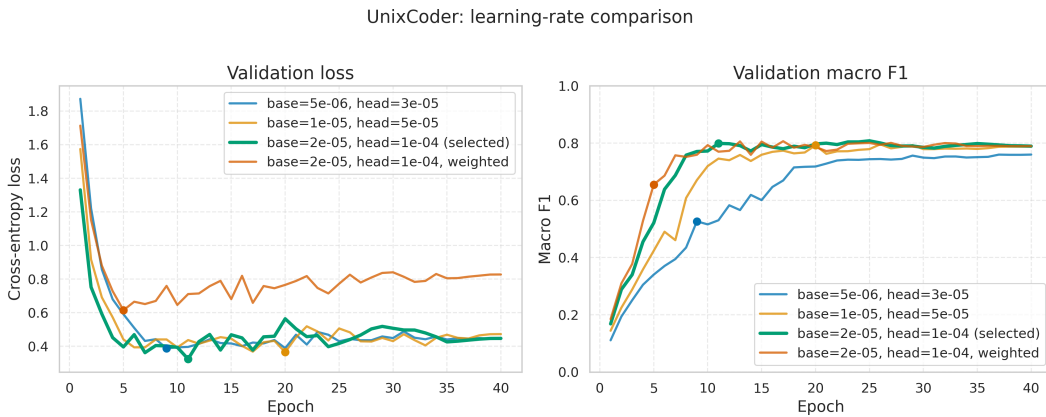


Fig. 6. Validation loss and macro F1 curves for UnixCoder across learning rates.

Table 3. Test performance and validation score for the trained NER models.

Model	Acc.	Prec.	Rec.	Macro F1	Fidelity	Val F1
BERT pre-trained	84.76	75.68	56.51	61.97	79.97	73.23
Naked BERT	73.95	67.68	41.08	44.17	75.06	56.53
UnixCoder	87.10	86.63	68.80	73.50	86.29	79.88
UnixCoder Last2+Head	83.89	75.36	61.11	64.85	83.05	72.24
UnixCoder Head Only	76.98	68.13	48.50	52.57	76.34	59.41

Model Comparison

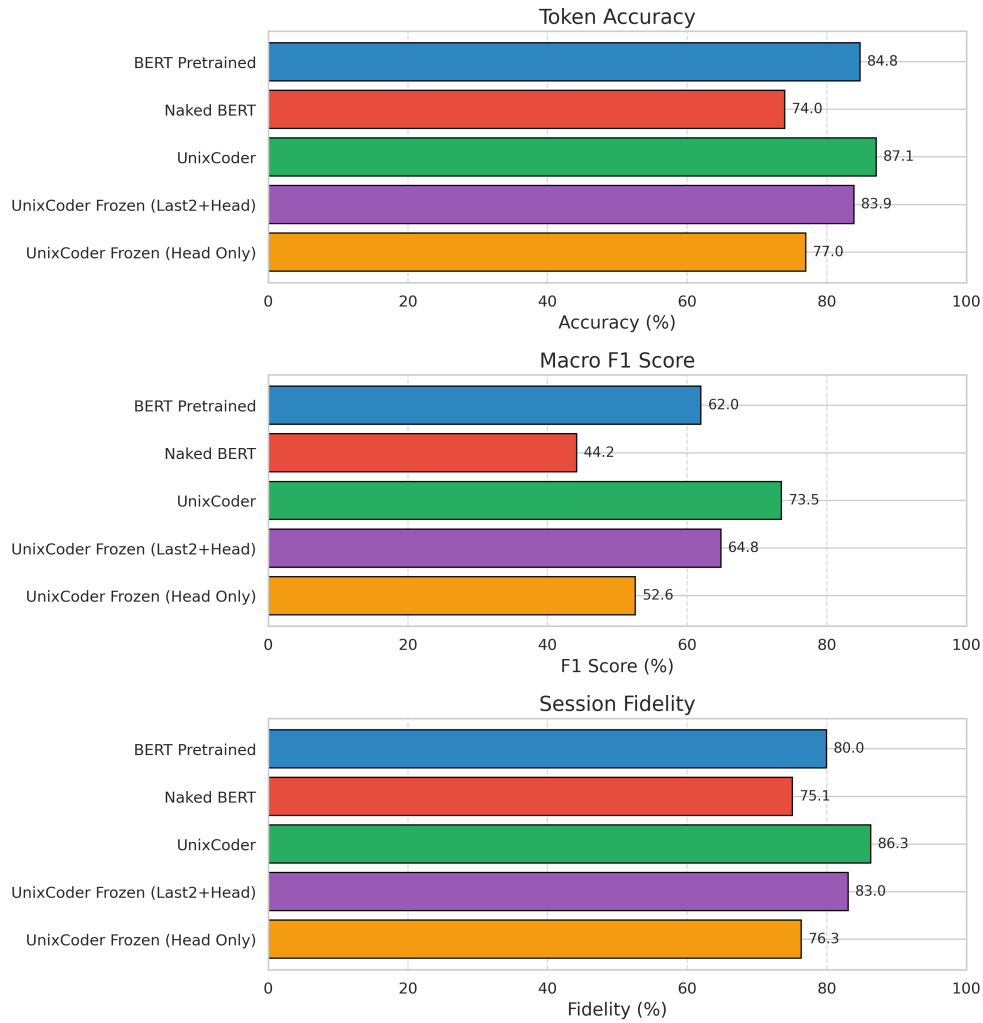


Fig. 7. Comparison of token accuracy, macro F1, and session fidelity across the trained models.

4.2 Per-Class Behavior

The per-class F1 plots in Figure 8 make the imbalance effect visible. Frequent tactics such as Discovery and Execution are learned more consistently, while classes with few examples remain less stable. This is the main difficulty of the task: high token accuracy can coexist with weaker balanced performance because most words belong to a small number of tactics. UnixCoder improves this behavior, but it does not remove the scarcity problem for the rarest classes.

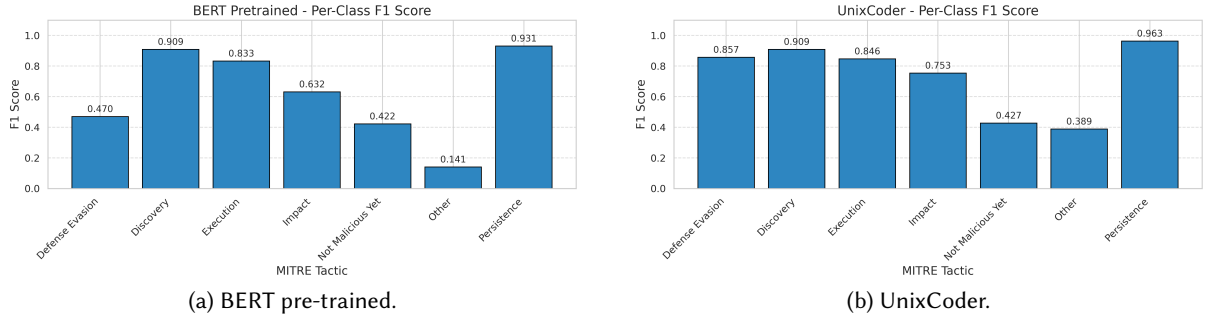


Fig. 8. Per-class F1 scores for the two fully fine-tuned pre-trained models.

The normalized confusion matrix in Figure 9 provides a more detailed view of the best model. The dominant classes are recognized reliably, which explains the high token accuracy and session fidelity. The remaining errors are concentrated in semantically close or low-support regions: some command chains with mixed behaviour are confused between Discovery and Execution, while the rarest tactics remain harder to separate because the model sees only a small number of labelled examples during fine-tuning.

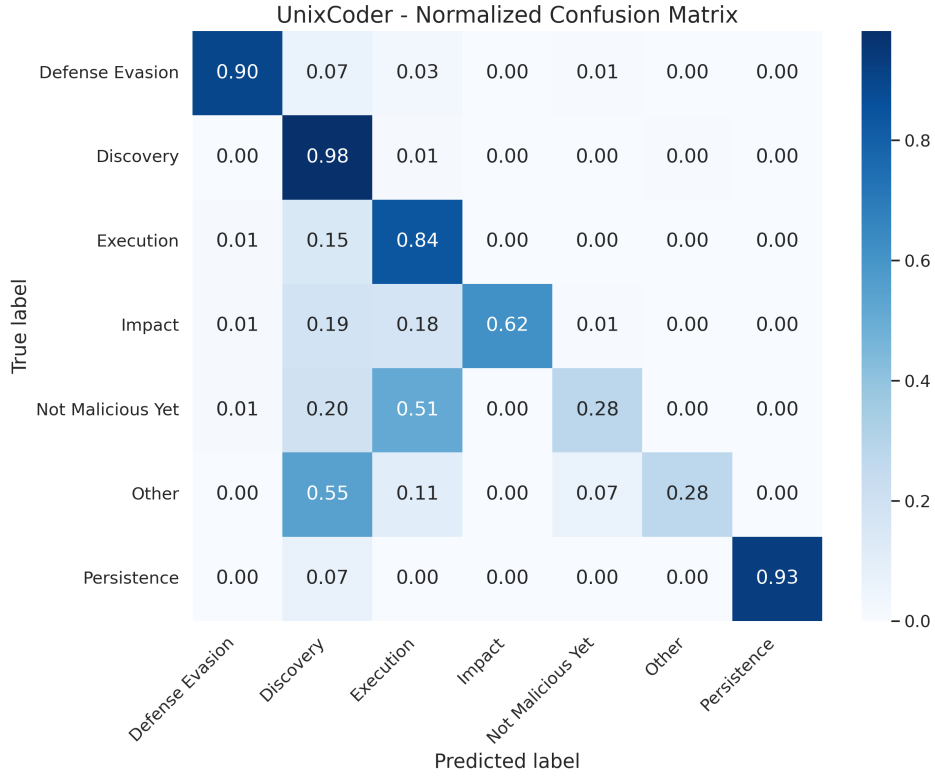


Fig. 9. Normalized confusion matrix for the fully fine-tuned UnixCoder model on the test set.

This error profile is coherent with the structure of the dataset. Many SSH sessions are long pipelines where a word can support more than one adversarial intent depending on the downstream command. The model is therefore not only learning command names, but also trying to infer the role of each word in a local execution context. The confusion matrix confirms that UnixCoder is the most reliable candidate, but it also motivates keeping per-class metrics in the evaluation rather than relying only on aggregate accuracy.

4.3 Frozen-Layer Experiments

The best full model is selected using validation macro F1 only; the selected backbone is UnixCoder. Two reduced fine-tuning strategies are then tested: training only the last two encoder layers plus the classification head, and training only the classification head. Table 4 summarizes the computational trade-off.

Freezing most of the encoder reduces trainable parameters from **125.3M** to **14.2M** and nearly halves training time, from **686.5 s** to **349.0 s**. However, macro F1 decreases from **73.50%** to **64.85%**. Training only the head is faster (**285.9 s**) and uses only **5,383 trainable parameters**, but the macro F1 falls to **52.57%**. Therefore, the pre-trained representation is useful, but adapting part of the encoder is still necessary for this domain. The

frozen settings also require larger learning rates than full fine-tuning, especially for the randomly initialized classification head.

Table 4. Training configuration and cost for the Task 3 models.

Model	Base LR	Head LR	Trainable	Epoch	Time (s)
BERT pre-trained	0.00001	0.00005	108,897,031	20	761.0
Naked BERT	0.00005	0.00005	108,897,031	13	764.0
UnixCoder	0.00002	0.00010	125,344,519	11	686.5
UnixCoder Last2+Head	0.00005	0.00025	14,181,127	16	349.0
UnixCoder Head Only	0.00100	0.00100	5,383	21	285.9

5 TASK 4: INFERENCE

The final stage applies the best validation model to the unlabelled `cyberlab.csv` sessions. The selected model is **UnixCoder**, which has the highest validation macro F1 among the fully trained and frozen configurations. The same word truncation used in Task 2 and Task 3 is applied before inference. Since the inference set is unlabelled, no test metric is computed; the model output is used only for qualitative and aggregate analysis. For each Bash word, the predicted tactic is taken from the **first sub-token**; if a session is truncated, only words that receive a prediction are considered.

The inference dataset contains **174,262 sessions**, spanning from **2019-09-01** to **2019-12-30**.

5.1 Command-Level Tactic Profiles

Table 5 reports the predicted tactic distribution for the required commands. The mapping is not one-to-one. `grep` is almost uniquely associated with **Discovery**, which is coherent with its common use for inspecting system information. `cat` is also mainly Discovery, but it appears as Execution when it belongs to command chains that decode or run payloads. `rm` is mostly Discovery or Execution, with smaller Defense Evasion and Persistence components. `echo` is the most context-dependent command: it is predicted as Persistence when it writes credentials or startup artifacts, as Execution when it feeds payloads to the shell, and as Discovery when used in benign-looking environment checks.

Table 5. Predicted MITRE tactic frequencies for selected commands in the inference set.

Command	Predicted tactic	Count	Freq. (%)
cat	Discovery	724,370	84.07
cat	Execution	137,283	15.93
cat	Not Malicious Yet	1	0.00
echo	Persistence	327,076	42.67
echo	Discovery	252,994	33.01
echo	Execution	186,177	24.29
echo	Other	190	0.02
echo	Impact	1	0.00
grep	Discovery	996,075	99.97
grep	Persistence	302	0.03
rm	Discovery	248,848	72.14
rm	Execution	82,502	23.92
rm	Defense Evasion	8,180	2.37
rm	Persistence	5,442	1.58

For every command-tactic tuple in Table 5, one representative session was inspected. The qualitative examples are consistent with the aggregate interpretation. For instance, `cat /proc/cpuinfo | grep name` is a Discovery pattern, while `echo <payload> | bash` is naturally closer to Execution. Similarly, `echo root:<password> | chpasswd | bash` supports Persistence because it modifies access credentials, and `rm -rf` inside cleanup chains can contribute to Defense Evasion.

The rare command-tag tuples are also informative. A very small number of `cat` and `echo` occurrences are assigned to minority labels such as Not Malicious Yet, Impact, or Other. These cases should not be interpreted as strong global associations; they are better read as local contextual decisions inside long command chains. For example, an `echo` used near destructive shell logic can be pulled toward Impact, while an `echo` embedded in a large encoded payload chain may be classified as Other when the surrounding context is ambiguous. This reinforces the main observation: the same Bash word can play different tactical roles depending on the neighbouring commands.

5.2 Fingerprint Analysis

Each session is converted into a fingerprint, defined as the ordered sequence of predicted MITRE tactics. The inference set produces **1,905 unique fingerprints**; **901** of them occur only once, indicating a long tail of rare behaviours. Fingerprints are sorted by date of birth, assigned increasing IDs, and counted per day.

Before looking at the temporal structure, the frequency distribution already shows that most fingerprints are rare, whereas a small number of recurrent patterns account for a large portion of the traffic. This is expected in honeypot data: many sessions are minor variants of automated scripts, while occasional payloads or command permutations create one-off fingerprints.

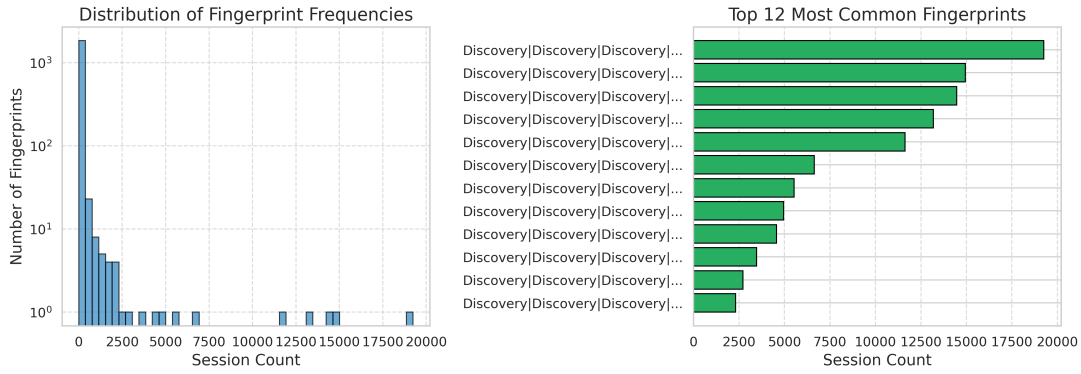


Fig. 10. Distribution of fingerprint frequencies and most common inferred fingerprints.

Figure 10 makes this imbalance explicit. The left panel highlights the long tail of low-frequency fingerprints, while the right panel shows that the most common patterns are dominated by repeated Discovery-heavy sequences. Therefore, a fingerprinting view is useful because it compresses a large inference set into a smaller number of behavioural templates.

Figure 11 shows the required time-vs-ID view. Older fingerprints appear near the bottom of the plot. No fingerprint is literally present during the entire **121-day** observation window, but some early fingerprints are highly persistent: IDs **0**, **11**, and **3** are active for 99, 98, and 96 days respectively. The most common fingerprint appears in **19,247 sessions**.

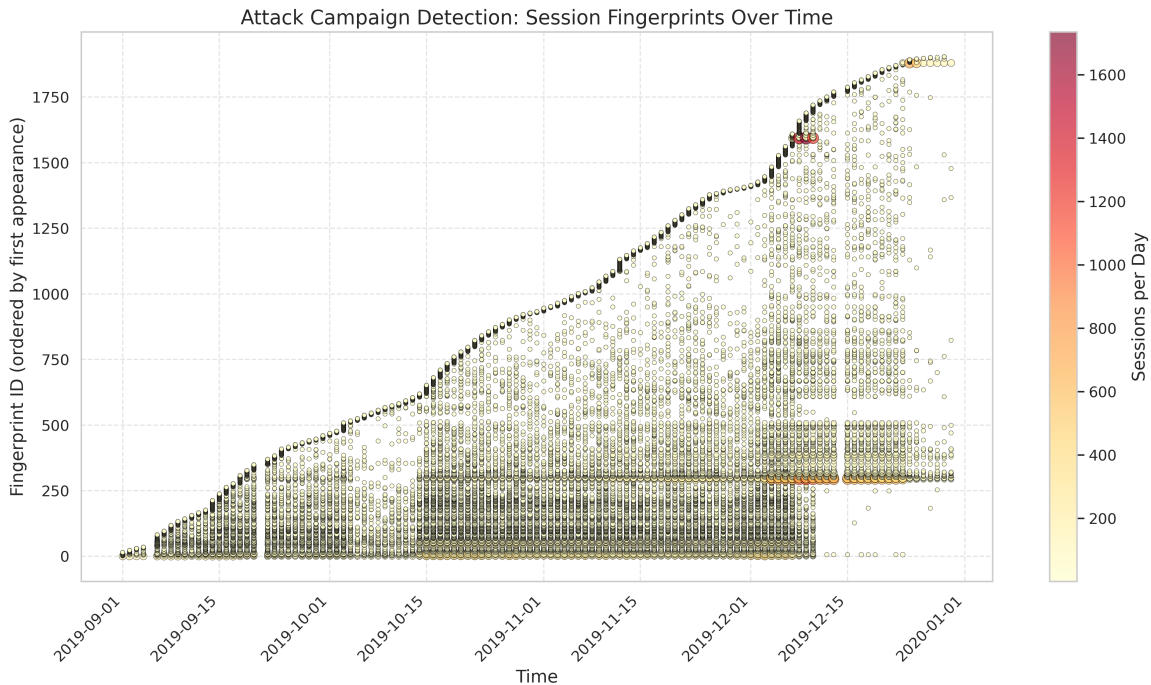


Fig. 11. Fingerprint IDs over time. Point size and colour encode the number of sessions assigned to a fingerprint on a given day.

The scatter plot also exposes concentrated bursts that are compatible with automated campaigns. The adaptive detector finds **6 campaign spikes**. The strongest burst is fingerprint **1594**, with **1,736 sessions** on **2019-12-09** and related spikes on 2019-12-08 and 2019-12-10. Fingerprint **293** shows a similar December concentration,

exceeding 900 sessions per day in the same period. These high-volume fingerprints include repeated Discovery chains such as CPU inspection followed by credential or payload-related commands, suggesting repeated scripted activity rather than isolated manual interaction.

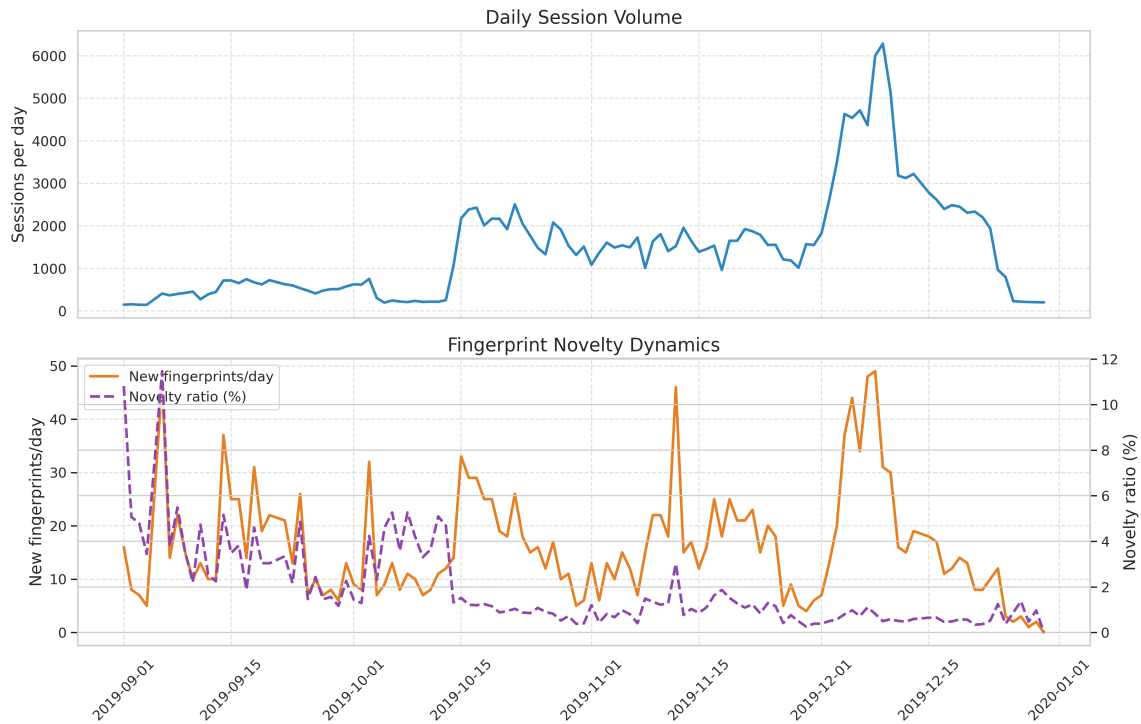


Fig. 12. Daily session volume and novelty dynamics for the inferred fingerprints.

Figure 12 provides a complementary aggregate view. The daily volume is not uniform: activity grows after mid-October and reaches its largest peak in early December, in agreement with the fingerprint spikes identified above. The novelty curve is higher at the beginning of the collection, when many fingerprints are observed for the first time, and becomes lower once repeated behaviours dominate the traffic. This suggests two different regimes: an initial phase with many newly observed patterns, followed by a period in which recurrent automated behaviours account for a large share of the sessions.

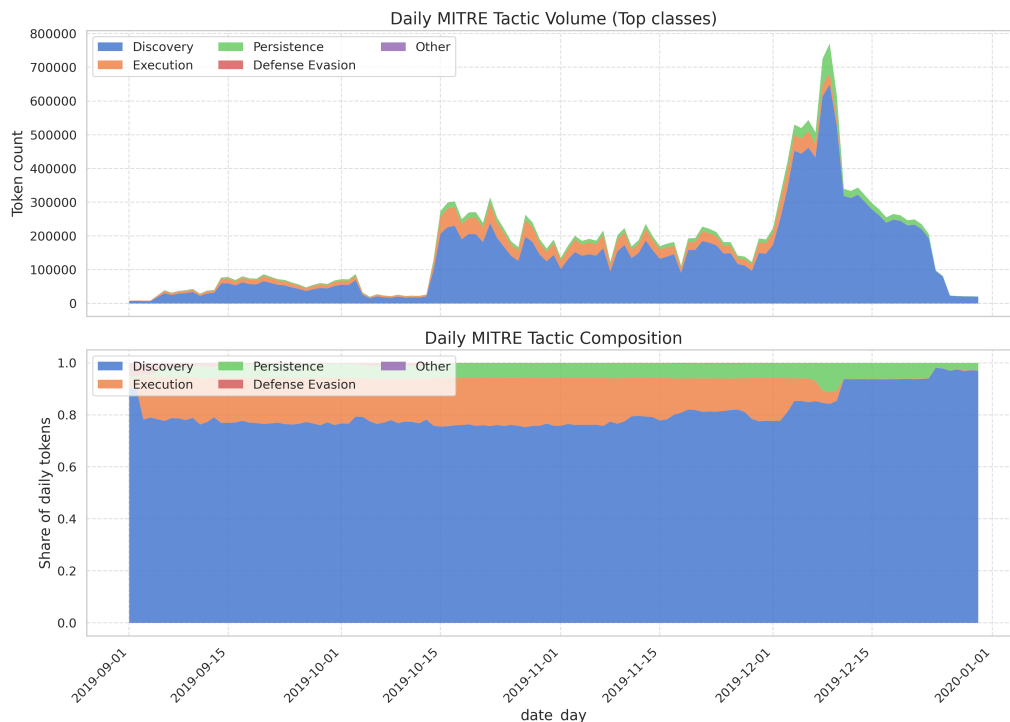


Fig. 13. Daily MITRE tactic volume and normalized tactic composition in the inference set.

The tactic-level view in Figure 13 confirms that the traffic volume is largely driven by Discovery-labelled words, but the relative composition is not perfectly constant. Execution and Persistence occupy visible shares during the central part of the observation window, while the strongest December peak is mostly a scale effect: many more sessions are observed, but the dominant tactical structure remains similar to the repeated fingerprints discussed above. This is useful from an analyst perspective because it distinguishes between a completely new behavioural mix and a large-scale replay of already observed attack patterns.

6 CONCLUSIONS

The laboratory confirms that SSH honeypot sessions can be treated effectively as contextual token sequences rather than as isolated command names. Tokenization is a central design choice: before preprocessing, long encoded strings create pathological sequences, while truncating words above 30 characters makes both BERT and UnixCoder usable under the 512-token limit.

In the NER stage, pre-training is clearly beneficial. Naked BERT remains well below pre-trained BERT, and the code-oriented UnixCoder model obtains the best overall performance, with the highest test macro F1 and session fidelity. The frozen-layer experiments show a practical trade-off: training only part of the encoder reduces computational cost, but full fine-tuning still provides the best balanced performance.

Finally, inference on CyberLab logs shows how the trained model can support security analysis. Command-level tactic profiles reveal that commands such as `echo` and `rm` are strongly context-dependent, while fingerprint analysis exposes persistent behaviours and high-volume December bursts that are compatible with automated attack campaigns.